How SysMon Parses the System

Where SysMon gets its system information, how it reads the ini and conf files, how it builds the port list, and how it decides whether the Pi's firewall lets the node's traffic through.

SYSTEM | PORTS | CONF AND INI | FIREWALL

SUMMARY · READ-ONLY REVIEW

Source: sysmon_pi02w_v6_13_83_20261028.py on branch ccr-9d4fe19f-ssud83. The full build's SysMon matches for every function named here.

What was read

SysMon gathers its data in one background loop, keeps the latest values in a shared state, and serves that state to the pages. Parsing happens at three points: when a value is read from the system, when a config file is read, and when a firewall or socket command's output is read.

1.1 Method

- Each parsing function was read in full, along with the functions it calls for its data.
- The full build's copy of each named function was compared by code structure. All 31 match the Pi Zero 2 W build, so the behaviour described here applies to both.
- Nothing was run on a node. Behaviour is described from the code.

1.2 The main loop

STEP	HOW OFTEN	WHAT IT DOES
System readers	Every 10 s	Hostname, kernel, uptime, load, memory, swap, CPU temperature, core voltage, throttling, disk, SD health, process count, wifimon's shutdown state.
Quick tests and port conflicts	Every third pass (30 s)	Each pinned service's state, its port, and its config file. Then the duplicate-port check.
Registrations	Every third pass, or until the state is OK	Live sign-in state for each network.
M17 reflector list	About hourly, if the file is stale	Refreshes the reflector list from its source.

The pages read the stored state. Requests that need a fresh answer (the Ports list, a probe, a firewall check) run their own command at the time of the request.

Where each value comes from

Most values come from a file under `/proc`, `/sys` or `/etc`, or from a short command. Each reader returns an empty value on failure, and writes a debug line only. A missing value shows as a dash on the page, not as an error.

VALUE	SOURCE	HOW IT IS PARSED
Uptime	`uptime -p`	The leading "up " is removed.
Load	`/proc/loadavg`	The first three fields, for 1, 5 and 15 minutes.
Memory and swap	`/proc/meminfo`	MemTotal and MemAvailable (and SwapTotal and SwapFree), converted from kB to MB. Used memory is total minus available.
CPU temperature	`/sys/class/thermal/thermal_zone0/temp`	Millidegrees divided by 1,000, to one decimal place.
Core voltage	`vcgencmd measure_volts core`	The number after `volt=`.
Throttling	`vcgencmd get_throttled`	The hex value after `throttled=`. Bits 0 to 3 mean "now" (under-voltage, frequency cap, throttled, temperature limit). Bits 16 to 19 mean "since boot". The status is "uv" if under-voltage is now, "throttled" if throttled now, "warn" if any "since boot" bit is set, otherwise "ok".
Disk	`os.statvfs("/")`	Percent used, using space available to ordinary users, and the sizes in GB.
SD health	`/proc/mounts`	The root filesystem's option list. "ro" means read-only, anything else "rw".
Process count	`/proc`	The number of entries whose names are digits.
Service state	`systemctl list-unit-files`, then `systemctl show`	Unit names must match a fixed pattern and end in `.service`. Installed units are then queried for their properties, and the process ID is matched back to its unit.
Sound cards	`/proc/asound/cards`	Parsed into a list of cards for the hardware page.
Serial and USB devices	`/sys/class/tty`, `/sys/bus/usb/devices`	ttyUSB and ttyACM devices, and the USB device tree.
Process name	`/proc/<pid>/comm`, `/proc/<pid>/cmdline`	Used to name the program behind a port.

2.1 What "unavailable" means

When a reader fails, it returns an empty value. For throttling, the page reads "vcgencmd unavailable" from an empty result, so a missing command and a healthy Pi look different. For the other readers, a failure looks like a blank. The book should say that a dash can mean either "not known" or "no data".

Listening ports

3.1 Reading the socket table

The port list comes from `ss -tlnpu`. Each line is parsed like this:

1. The header line is skipped.
2. The protocol (first field) must be `tcp` or `udp`.
3. The local address (fifth field) is split at its last colon. The port must be all digits, so the address and the port are kept apart correctly for IPv6 forms.
4. The process ID comes from `pid=`, and the process name from the first quoted word after it.

The list shows each protocol and port once (the first entry wins), sorted by port number. A port bound on both IPv4 and IPv6 therefore appears once.

3.2 Labels from the pinned services table

The labels in the port list come from the pinned services table. Each row has five fields: unit, description, config file, port and protocol. Rows that begin with `__GROUP__:` start a group. The default table names these groups: System, ASL-DVS, Asterisk Core and M17. The default table gives SSH port 22, Cockpit 9090, the Dashboard 8989, SysMon 9999, Asterisk 4569/udp, and Allmon3 8080.

A label is attached by port number only. If two units share a port, the later one in the table wins. A row whose port is `-`, `parse` or blank has no label.

3.3 The `parse` value is not used

The table's port field accepts the word `parse`, which seems meant to read the port from the unit's config file. No code does this. The quick tests and the service list skip `parse`, and show it as a dash. A row written as `parse` therefore never gets a port. The book should not describe this as working.

3.4 Duplicate ports

Every 30 seconds, a port is reported as a conflict when at least two different process IDs are listening on the same protocol and port. Two sockets from one process do not count.

3.5 Quick tests

For each pinned unit, the quick test checks four things in order: the unit is installed, it is active, its port is listening, and its config file exists and is not empty. The result is pass, fail or not applicable.

Caution. The listening check is a substring test: it looks for the text `:port` anywhere in the `ss` output. Port 443 would therefore also match a listener on 4430. The duplicate and probe code uses a different, exact test (see 3.6), and the quick test should match its style.

3.6 On-demand probes

The probe page tests one port over TCP, HTTP or UDP. It builds its target list from the addresses that `ss` reports for that port, then adds loopback (127.0.0.1 and ::1) as fallbacks. Addresses of `0.0.0.0` and `*` are replaced by loopback. A link-local IPv6 address keeps its interface scope. Nothing is probed beyond the node itself.

The phone check's UDP listener test (not the probe page) uses an exact suffix match on `:port`, so it does not match a longer port. This is the safer form.

Reading the conf and ini files

4.1 Which files are opened, and how

- **Asterisk conf files.** A file is accepted only if its name matches `^[\w\-]+\.conf$` and holds no slash or `..`. It is read from `/etc/asterisk`.
- **Other ini files.** These are opened by an explicit list of paths, tried in order. Examples: Analog_Bridge.ini in `/opt` or `/etc`; USRP2M17.ini in `/opt/USRP2M17`; DVSwitch.ini, MMDVM_Bridge.ini and the YSF, P25 and NXDN gateway ini files in `/opt`, `/etc` and `/etc/dvswitch`; allmon3.ini in the Allmon3 folder.
- **Size.** Any file is read up to 64 KB. Beyond that, the text is cut and a line says so. A key near the end of a large file can be missed.
- **Decoding.** Files are read with errors replaced, so a bad byte does not stop the read.

4.2 The parsers

FILE	PARSER	WHAT IT TAKES OUT
rpt.conf	`_rpt_scan_full`	Node sections and their keys. It understands templates (`[name](!)`), inheritance (`[1999](node-main)`), and `;--` to `--;` comment blocks. It records which section each key came from, and lists any template that is missing. A node is a numeric section that is not on the reserved list.
rpt.conf (USRP lines)	`parse_rpt_conf`	The RX and TX ports from a `USRP/host:rx:tx` channel line.
iax.conf	`_dvsm_read_iax_conf`	The `[general]` bindport (default 4569) and bindaddr.
pjsip.conf	Phone context loader, then `_ph_transport_for`	The first UDP transport and its bind address. The port after the last colon is the SIP port. The default, if none is set, is 5060.
rtp.conf	`_ph_rtp_range`	`rtpstart` and `rtpend`, with comments removed. If Asterisk is running, `rtp show settings` is used instead.
manager.conf	`parse_manager_conf`	Each user section other than `[general]`. It reports whether a secret is set, and never shows the value.
extensions.conf	`_ph_build_dialplan`	The `exten` and `same` lines, turned into a dial plan.
Analog_Bridge.ini	`_dvsm_read_ab`	The rxPort and txPort in the `[usrp]` section.
USRP2M17.ini	`_m17_read_config`	LocalPort, DstPort and gain from `[M17 Network]`, and the USRP ports from `[USRP Network]`.
DVSwitch.ini	`parse_dvswitch_ini`	txPort and rxPort for each section (DMR, D-STAR and others).
DVSwitch var.txt	`parse_dvswitch_var_txt`	Network entries, shown as address:port.
allmon3.ini	`parse_allmon3_ini`, `parse_allmon3_web`	Node sections, and the web port.
Dashboard conf, phone.json	Phone context loader	The dashboard's HOIP server and port.

The parsers that were not traced line by line are echolink.conf, savenode.conf, rpt_http_registrations.conf, modules.conf and the MMDVM gateway log settings. Their names and inputs are known, but their parse rules are not described here.

4.3 How the section parser works

- A section header is a line in square brackets. A key is `name = value`. In the rpt.conf parser a semicolon ends the value; in the Phone parser an escaped `\;` is kept and only an unescaped one starts a comment.
- Key case differs by parser. The rpt.conf and Phone parsers lower-case keys. The DVSwitch and USRP2M17 parsers keep the original case. A search by key must match the parser's case.
- The generic `_ini_sections` function takes the header and key patterns as parameters, so each file type can define its own rules.

4.4 The Phone context loader

The phone checks read the Asterisk files as one set. `_ph_parse_file` follows `#include` and `#tryinclude` lines, up to three levels deep. Each file must be 1 MB or smaller, and its name must match `^[\w.\-]+$`. Lines between `; >>> dvs-phone` and `; <<< dvs-phone <<<` are marked as managed by the dashboard, as are whole files named `dvs_phone_*`.

Deciding whether the Pi lets traffic in

The firewall check does not read the whole firewall. It asks one question: are the UDP ports the node needs open to incoming traffic? The two ports it needs are the SIP port and the RTP audio range.

5.1 Which firewall is found

ORDER	TEST	NOTE
1	`firewall-cmd --state` says running	firewalld
2	`ufw` is installed	Installed is enough. Whether ufw is active is checked later.
3	`nft` is installed	nftables
4	`iptables` is installed	iptables
5	None of the above	"no firewall on the Pi"

5.2 Reading the rules

- **firewalld.** Each active zone is read with `--list-all`. If a zone's target is ACCEPT, everything is open. Otherwise, the zone's ports are read, its services are expanded from their XML files (in `/etc/firewalld/services` or `/usr/lib/firewalld/services`, following includes up to four levels), and its rich rules are matched for port, protocol and accept, reject or drop.
- **ufw.** `ufw status verbose` is checked first. If ufw is not active, the check stops. If the default policy allows incoming traffic, everything is open. Otherwise `ufw status numbered` is read. IPv6 rules are skipped. ALLOW and LIMIT rules count.
- **nftables.** The ruleset is read. If every input chain accepts everything, the result is "open". Otherwise the check says it cannot read the rules port by port.
- **iptables.** The INPUT chain is read. If the policy accepts and there is no DROP or REJECT, the result is "open". Otherwise the check says it cannot read the rules port by port.

5.3 The coverage check

Only UDP rules that allow traffic are collected. The needed range (one port for SIP, or the RTP range) is compared with them, and any part not covered is listed as a gap. The result is one of five states:

STATE	MEANING
open	Every port in the range is allowed.
partial	Some ports are allowed. The gaps are listed.
closed	None of the range is allowed (the whole range is a gap).
nofilter	The firewall lets everything in (firewalld target ACCEPT, ufw default allow, or an accept-all chain), or ufw is not active.
unknown	The firewall's rules cannot be read port by port (nftables or iptables with filtering).

5.4 Fix advice

The advice names the port by taking the last word of the check's label, then gives the command for the firewall found: a firewalld command, a ufw allow rule, an nftables reload, or a general instruction when none is found.

5.5 What the check does not cover

- Only UDP. The TCP ports in the pinned table (Dashboard, Cockpit, Allmon3, SSH) are not checked against the firewall.
- IAX2 on 4569/udp is not checked, although it is a UDP port that the node uses. The coverage check has only two entries: SIP, and audio.
- Only the phone checks call the coverage code. The Ports tab does not show firewall state beside each port.

What the parsing shows

6.1 A rule's source is ignored

In the firewall check, a rule counts as "open" if it allows UDP, whatever its source. A firewalld rich rule that accepts UDP from one address, and a ufw rule that allows UDP from one subnet, are both counted as open to everyone. The source field is read, but never used. A node could therefore show a port as open when only a single address can reach it. Fix this before the book describes the check as a test of reachability.

6.2 Substring port match in the quick test

The quick test finds a listener by searching for `:port` anywhere in the output. Port 443 would match 4430. The exact match used elsewhere (`endswith`) avoids this, and the quick test should use it too.

6.3 The `parse` port is not implemented

The pinned table advertises a way to read a port from a config file, but the code never does it. A user who writes `parse` gets a dash and no test. The book should either describe the port table as manual, or the code should be finished.

6.4 Failures are silent

A reader that fails returns an empty value and logs only at debug level. The page then shows a dash. A missing command, a permissions problem and a healthy empty value look the same. The book should tell the operator where to look (the debug log) when a value is always blank.

6.5 Truncated files

A conf file over 64 KB is cut, and the parsers see only the first part. This is unlikely for the usual Asterisk files, but a large rpt.conf or a long dial plan would lose its tail without a warning on the page.

6.6 Two ways to match a port

The quick test uses a substring match, and the phone check's UDP listener test uses an exact suffix match. The port list uses a third method: it parses the full socket table and keeps the first entry for each port. Pick one for the book, and note that the substring form can report a listener that is not there.

6.7 Duplicate labels

Two pinned units on the same port give one label, the later one. A second unit on that port is not shown in the list, so an accidental clash may not be noticed until the duplicate check reports it.

What this summary does not cover

- The parse rules of echolink.conf, savenode.conf, rpt_http_registrations.conf, modules.conf and the MMDVM gateway log settings were not traced line by line.
- The exact output of `ss` and `ufw` on a live node. The parsers were read against their expected formats, not tested on real output.
- The firewalld rich-rule expression and its variants, beyond the port, protocol and verdict the code reads.
- How the pages display the values. That is covered in the HTML and JavaScript summaries.